# AI Usage Log: Sprint 2

SOEN 341: CareerConnect

Youssef Karnib, Backend Team

**AI Tool Used:** ChatGPT (OpenAI).

**My Sprint 2 Work:** Job listing, search and detail endpoints (YO-1); recruiter posting and applicant endpoints (YO-3); and application status tracking (YO-2).

## Task 1: Job Listing, Search and Details (YO-1)

**Purpose of AI Use:** Code generation, understanding the existing backend, and endpoint testing.

**Chat Link or Prompt/Response:** Prompt (paraphrased): “Guide me through implementing YO-1 in VS Code using Kaila’s job search functions, and explain how to test it in Swagger UI.” Response: ChatGPT inspected code I supplied, suggested the schema and router files, and walked me through authentication, testing and the GitHub submission steps.

**AI-Suggested Content:** Job list and detail response schemas; GET /api/v1/jobs with keyword, location, category and pagination; GET /api/v1/jobs/{id}; a JWT current-user helper; and router registration in main.py. The implementation reused the team’s existing search_jobs and get_job functions.

**Validation:** I logged in through Swagger, confirmed that the jobs endpoint responded, created a sample posting through the recruiter endpoint, and checked that it appeared in job search and detail results. I also ran the existing backend and frontend tests.

**Decision:** Modified before use. I preserved the registration endpoint’s HTTP 400 validation behavior while supporting HTTP 422 for newer endpoints so existing tests would keep passing.

**Reflection:** I learned how FastAPI dependencies protect endpoints, why the search router should reuse existing database functions, and how to test authenticated requests with Swagger.

**Responsible Person:** Youssef Karnib

## Task 2: Recruiter Endpoints (YO-3)

**Purpose of AI Use:** Code assistance, recruiter authorization, Swagger testing, and GitHub workflow guidance.

**Chat Link or Prompt/Response:** Prompt (paraphrased): “Help me implement recruiter job creation and an applications list, while ensuring recruiters can only access their own postings.” Response: ChatGPT suggested the schemas and router, explained role and ownership checks, and guided me through testing with recruiter and job-seeker accounts.

**AI-Suggested Content:** POST /api/v1/recruiter/jobs to create an open posting; GET /api/v1/recruiter/jobs/{id}/applications to list applicants; validation of required fields and categories; and permission checks that reject job seekers and prevent recruiters from viewing other recruiters’ applicants.

**Validation:** I created a test posting in Swagger, verified it appeared in GET /jobs, and checked that the new posting returned an empty applications list before anyone applied. The existing test suite passed after the shared validation handler was adjusted.

**Decision:** Accepted with integration changes. I retained the existing job and application models and kept YO-1 and YO-3 in separate Git branches and pull requests for review.

**Reflection:** I better understood role-based access control and why checking ownership is essential even when someone is already authenticated as a recruiter.

**Responsible Person:** Youssef Karnib

### Task 3: Application Status Endpoints (YO-2)

**Purpose of AI Use:** Code generation, database migration guidance, debugging, and workflow testing.

**Chat Link or Prompt/Response:** Prompt (paraphrased): “Help me implement My Applications and recruiter status updates using Lamees’s apply endpoint, and fix the database and test errors.” Response: ChatGPT suggested the new router, schema additions and migration, helped interpret Alembic output, and guided me through Swagger and pytest verification.

**AI-Suggested Content:** GET /api/v1/applications/me for applications owned by the logged-in user; PATCH /api/v1/applications/{id}/status for the job’s recruiter; the five required statuses; an updated_at column; and an Alembic migration replacing the old submitted default with Applied.

**Validation:** I applied the migration successfully to my local SQLite database, then uploaded a resume, submitted an application, changed its status to Interview as the recruiter, and verified that the job seeker saw Interview. I fixed an older test that still expected submitted; the backend test run then reported 72 passed.

**Decision:** Modified before use. I reused the team’s shared authentication and status definitions, preserved Lamees’s application submission endpoint, and updated the outdated test expectation to match the new default.

**Reflection:** I learned why changing a SQLAlchemy model does not update an existing database, how Alembic migrations work, and how integration tests catch differences between an old schema and a new feature.

**Responsible Person:** Youssef Karnib